# zRA 详细逐页演讲稿

## 原版适度扩充 · 主讲与扩展讲解

本稿逐页对应原PPT的20页；页数、顺序和原有图示保留，7页仅修改25处说明文字。每页分为“主讲段”和“扩展讲解”，两者共同写入PPT演讲者备注。逐页正文共16,421字符，其中约12,877个汉字，不含附录。

20分钟时间表只用于主讲段与图表讲解，建议通过演练调整。完整扩展稿如全部朗读，预计约45–60分钟，实际受英文符号、停顿与语速影响；应作为备讲材料选用。章节分隔页用简短过渡，不必展开全部补充段落。

论文和复现分别标注：原PPT第16页截图是已有实验结果，本次未重新运行zRA实验。文末提供答辩问答、数据核对和逐项修改记录。

# 20分钟主讲时间表

| 页 | 主题 | 参考用时 | 累计至 |
|---|---|---|---|
| 1 | 封面：从交互走向独立 | 25秒 | 00:25 |
| 2 | 目录：研究背景、设计、实验与评价 | 20秒 | 00:45 |
| 3 | 研究背景与问题 | 10秒 | 00:55 |
| 4 | 场景驱动：任意第三方独立验证设备 | 55秒 | 01:50 |
| 5 | 传统RA依赖特定验证者 | 55秒 | 02:45 |
| 6 | 传统RA与zRA的验证流程区别 | 60秒 | 03:45 |
| 7 | zRA设计章节过渡 | 10秒 | 03:55 |
| 8 | zRA的三大组件 | 80秒 | 05:15 |
| 9 | zRA的四个阶段 | 85秒 | 06:40 |
| 10 | Setup：未来响应承诺与两层Merkle树 | 105秒 | 08:25 |
| 11 | Attestation：证明关系、公开输入与私有见证 | 120秒 | 10:25 |
| 12 | Verification：四项接受条件 | 90秒 | 11:55 |
| 13 | 安全性：攻击方式与安全边界 | 75秒 | 13:10 |
| 14 | 实验章节过渡 | 10秒 | 13:20 |
| 15 | 实际应用架构与工作流程 | 75秒 | 14:35 |
| 16 | 设备端证明：论文结果与已有复现 | 150秒 | 17:05 |
| 17 | 优势：制造商通信与全网成本 | 80秒 | 18:25 |
| 18 | 论文评价与启示章节过渡 | 10秒 | 18:35 |
| 19 | 结论：贡献、局限与启示 | 75秒 | 19:50 |
| 20 | 致谢与提问 | 10秒 | 20:00 |

# 第01页 封面：从交互走向独立

主讲参考：25秒

## 主讲段（20分钟参考）

大家好，我们今天汇报的是 NDSS 2024 的论文《From Interaction to Independence: zkSNARKs for Transparent and Non-Interactive Remote Attestation》。这篇论文提出一种叫作 zRA 的远程证明协议，希望让任何第三方都能独立检查设备的软件完整性，而不必先获得设备的秘密，也不必向某个可信服务器询问验证结果。我们会沿着原 PPT 的结构，先说明这个问题为什么重要，再介绍协议如何实现公开验证，最后讨论原型实验以及它的适用边界。

## 扩展讲解（按需选用）

可以先用一句比较直观的话理解论文题目中的变化：传统方案经常需要“找特定的人来问这台设备是否可信”，而 zRA 希望把设备的证明变成“拿到公开证据以后，自己就能检查”。这里的独立指的是验证者可以独立完成验证，不代表设备与网络完全没有通信，也不代表整个系统没有任何信任前提。

这篇论文由 Shahriar Ebrahimi 和 Parisa Hassanizadeh 完成。我们汇报时把重点放在设计逻辑上：它到底解决了传统远程证明中的哪一种依赖，为什么需要零知识证明和承诺树，以及为了获得这种验证能力，设备和基础设施分别付出了什么代价。带着这些问题阅读后面的图，比单独记住一组密码学名词更容易理解论文。

内容出处：论文首页、摘要；原PPT第1页

# 第02页 目录：研究背景、设计、实验与评价

主讲参考：20秒

## 主讲段（20分钟参考）

今天的汇报分为四部分。第一部分介绍研究背景与问题，重点看传统远程证明为什么仍然依赖特定验证者。第二部分是 zRA 的设计，依次说明三类密码学组件、四个协议阶段，以及证明端和验证端的具体工作。第三部分结合论文实验与原 PPT 中已有的复现截图，分析耗时和可扩展性。第四部分总结贡献、成本和启示，回答它在什么条件下适合使用。

## 扩展讲解（按需选用）

这四部分之间的联系，可以概括为“ 问题、机制、证据、判断 ”。我们先提出一个要求：普通参与者也应该能够检查设备证明。随后看 zRA 怎样满足这个要求，而不仅仅是怎样生成一个数学上有效的 proof。实验部分再检查这样的设计能否在实际平台上运行，最后把实验结论放回网络场景中，判断它能解决什么、还不能解决什么。

原 PPT 以二十分钟汇报为目标，现场主讲可以采用每页的主讲段。这里额外准备的扩展讲解，是为了帮助理解原理和应对提问；如果逐字讲完所有扩展内容，需要增加时间。对于章节分隔页，我们做简洁过渡，把更多时间留给第十至十三页的协议细节，以及第十六、十七页的实验解释。

内容出处：原PPT第2页；论文I – IX

# 第03页 研究背景与问题

主讲参考：10秒

## 主讲段（20分钟参考）

先进入研究背景。远程证明已经能够帮助远端检查设备的软件完整性，但论文指出，验证能力往往只掌握在少数具有特权信息的参与者手中。因此这里真正要讨论的，是怎样把验证权限从特定验证者扩展到普通第三方，同时减少每次证明对在线可信服务的依赖。

## 扩展讲解（按需选用）

“ 设备可信 ”是一个很宽泛的说法，后面需要把它收紧到具体的测量对象和安全假设。远程证明通常检查指定的软件、内存或者配置是否符合认可状态，它不能仅凭一个证明就保证设备今后永远安全，也不能保证所有业务逻辑没有漏洞。

因此，理解这部分背景时要同时看两个层面：一个层面是设备怎样产生与自身真实状态绑定的证据；另一个层面是谁有能力判断这份证据是否有效。zRA 主要改变第二个层面的验证方式，但它仍然需要第一个层面具有可靠的基础。

内容出处：原PPT第3页；论文I、III

# 第04页 场景驱动：任意第三方独立验证设备

主讲参考：55秒

## 主讲段（20分钟参考）

这一页用 IoT 设备解释需求。假设一台智能门锁、车载终端或者其他联网设备，需要向云服务证明自己运行的是认可的软件。设备制造商可能知道它的初始配置，但合作方和普通用户未必拥有这些信息。如果第三方只能查询制造商给出的“ 通过 ”结果，那么它仍然在信任制造商的判断，而没有独立核验证据。zRA 的目标，是让设备提交一个能与公开参数绑定的证明，云服务、合作方和用户都能自行检查。这里不公开的是证明中的响应与成员路径等见证，设备身份、当前挑战和认可根仍然需要公开绑定。

## 扩展讲解（按需选用）

可以沿着页面上的四个步骤来讲。第一步是出现验证需求：某个服务要决定是否接受设备接入，或者用户希望检查一个远程终端的软件状态。第二步是设备生成证据：它根据当前挑战计算响应，再把这个响应放进零知识证明的见证中。第三步是第三方公开验证：第三方不需要得到设备秘密，只要拿到证明、公开输入和验证参数，就能检查证明是否成立。第四步才是应用层采用这个结果，例如决定是否允许设备继续参与服务。

原图里“ 请求设备证明 ”的箭头可以理解为应用场景的触发方式。协议本身允许设备获取一个全局挑战以后主动生成证明，并不要求每个验证者都单独发起一轮挑战—响应会话。也就是说，一个应用有接入请求，不会自动使底层证明变回逐验证者的交互式协议。需要区分业务流程中的请求与证明系统必须进行的交互。

此外，图中的“ 设备在本地执行可信计算 ”需要进一步解释。设备真实状态的测量与响应生成由最小信任锚保护；生成 zkSNARK 证明的普通软件层则可以是不可信的。零知识证明负责检查见证满足承诺和路径关系，不能把一个虚假的测量输入自动变成真实状态。我们随后会用 ROM 和 MPU 的分工说明这一点。

这一页列出的隐私、公开验证和扩展性，是论文希望同时获得的能力。但公开验证并不意味着匿名：设备公钥或原型中的地址仍然作为身份参与验证。隐私也不意味着平台看不到证明时间、设备身份或提交记录。因此，准确的表述是减少对内部测量与响应见证的直接暴露，而不是声称隐藏了设备的一切信息。

内容出处：原PPT第4页；论文I、III、IV-A及Fig.4

# 第05页 传统RA依赖特定验证者

主讲参考：55秒

## 主讲段（20分钟参考）

这一页把传统方案的限制归纳为四点。交互依赖意味着一次证明常常需要一个掌握参考信息的验证者参与挑战和响应检查；机密依赖意味着它可能需要预共享密钥或者初始测量。设备数和证明次数增加以后，这种逐设备的处理又会带来扩展压力。最后，即使系统公布了验证结果，普通参与者也未必能自行检查产生结果的过程。论文希望让验证所需的依据公开，第三方不必依靠持有设备秘密的主体。这里是对论文所讨论方案的概括，不应把四个问题机械地套到所有远程证明协议上。

## 扩展讲解（按需选用）

交互和机密依赖是两个相关但不同的问题。一个方案即使支持异步提交，也可能仍然需要某个代理或者可信服务在后台处理设备秘密，然后向外公布结果。反过来，一个方案也可能允许多个参与者检查某些信息，但还保留每次证明中的专门交互。所以，评估一项改进时，不能只看它是否把请求写进了区块链，或者是否支持异步通信，还要看最终判断是不是能够由公众直接重算。

论文讨论了 SCRAPS、PROVE 等已有工作。SCRAPS 将部分验证工作交给私有智能合约和代理结构，在 Pub/Sub 场景中改善了异步处理；PROVE 使用单向密钥链等机制提供公开可验证的证明设计，但仍有事件捕获、日志和基础设施方面的要求。这些方案已经取得了不同方向的进展，zRA 强调的是进一步把最终验证变成公开的证明检查。

扩展性差也不能简单理解成“ 传统设备不能很多 ”。具体瓶颈来自谁在每个周期为多少设备处理请求，以及服务是否一直在线。例如，若一个中心服务必须逐个接收设备请求并计算预期响应，那么增加设备数量会增加这个服务的负担。一旦服务故障，依赖它的设备证明也受到影响。

因此这一页的目标不是否定传统远程证明，而是指出一种可以改善的组织方式：将原来由少数验证者完成的特权检查，改造成基于公开参数的密码学证据检查。这样改变的是验证角色和在线路径，设备端测量本身仍然需要可信基础。

内容出处：原PPT第5页；论文I、III、VIII及Table II

# 第06页 传统RA与zRA的验证流程区别

主讲参考：60秒

## 主讲段（20分钟参考）

左边是传统挑战—响应流程：验证者发送新挑战，设备根据自身状态返回响应，验证者利用设备密钥、参考测量等信息判断响应是否正确。右边的 zRA 把“ 取得新挑战 ”和“ 检查证明 ”拆开：制造商按周期公布一个全局挑战，设备取到挑战以后自主生成证明，任何人都可检查公开证据。因而非交互指的是不需要每个验证者和每台设备完成一轮专门的交互，也不需要每次向制造商询问正确响应。设备仍须访问最新挑战、取得所需路径并提交证明；制造商也仍须保证挑战按时发布。

## 扩展讲解（按需选用）

从左侧箭头可以看到，传统方案中 challenge 的作用是保证证据与这次验证相关，避免设备重复提交以前的正确结果。设备响应还必须依赖真实测量，否则它只要记住一个固定的“ 正确答案 ”就能蒙混过关。验证者之所以是特定参与者，就是因为它需要判断当前状态下应该出现什么响应。

右侧并没有取消新鲜性，而是把随机挑战变成全局共享。一个周期内，各设备使用同一个挑战；设备身份又被放进响应承诺，所以设备之间仍然不能混用证据。不同验证者可以检查同一份证明，不必分别给设备发一个挑战，也不必分别获取设备秘密。

如果有人问“ 制造商难道完全可以下线了吗 ”，答案要分阶段。初始化以后，制造商不需要逐设备处理在线证明请求，但它或者挑战分发参与方还需要持续发布下一个挑战。它也需要处理软件更新涉及的认可根和参数维护。论文减少的是每次证明中的逐设备在线服务，而不是取消系统生命周期中的全部管理职责。

这一页还有一个容易混淆的概念：零知识证明并不是直接把设备的内存展示给公众，然后让公众重新测量。公众看到的是一个证明，证明者知道一个响应，并且该响应与设备身份、挑战和认可根之间满足约束。只要测量信任锚可靠、初始化承诺正确、验证条件完整，这种成员关系证明才有远程证明的语义。

内容出处：原PPT第6页；论文I、IV-A、Fig.2及Alg.2 – 3

# 第07页 zRA设计章节过渡

主讲参考：10秒

## 主讲段（20分钟参考）

接下来进入 zRA 的设计。我们先理解 zkSNARK、Merkle Tree 和承诺分别承担什么工作，再沿着 Setup、全局挑战、Attestation 和 Verification 四个阶段，说明它们如何连接。设计部分最关键的问题是：设备证明的究竟是哪一个关系，第三方接受证明时又必须检查哪些条件。

## 扩展讲解（按需选用）

这一部分可以带着三个具体问题听。第一，正确响应不能直接交给每个验证者，那系统用什么方式提前认可它。第二，设备如何在不把响应放进公开输入的情况下，证明自己的响应被认可。第三，如果攻击者自己构造了一棵树，或者使用旧挑战生成了一个数学上有效的证明，验证端怎样拒绝它。

这三个问题分别对应承诺、成员证明和外围验证条件，也基本解释了后面几页的安排。把这条逻辑线连起来，比把四个阶段当作互不相关的接口更有帮助。

内容出处：原PPT第7页；论文II、IV

# 第08页 zRA的三大组件

主讲参考：80秒

## 主讲段（20分钟参考）

zRA 的三类组件各有分工。zkSNARK 让证明者证明自己知道满足约束的见证，具有简洁证明和高效验证的特点。Merkle Tree 把大量被认可的承诺组织成一棵树，用一个根表示整组承诺，设备只需使用对应路径完成成员证明。承诺则把设备身份、正确响应和挑战绑定到一个哈希叶子中。实际 zRA 的树叶是未来 challenge-response 组合的承诺，不是把所有原始状态直接放进树。三个组件组合后，公众不需要看到响应或整条路径，就能检查响应承诺属于认可根。

## 扩展讲解（按需选用）

先看 zkSNARK。它可以被理解为一个针对明确计算关系的证明工具：证明者拿到公开输入和私有见证，证明这些值满足电路约束。零知识表示证明不会直接展示私有见证；可靠性要求错误的见证不能轻易通过验证；简洁性则使验证者不必重新执行证明者所做的全部计算。这里“零知识”不等同于计算加密，公开输入仍然会公开，见证也不是因为被放进电路就自动具有可信来源。

再看 Merkle Tree。叶子经过哈希，两两组合得到父节点，一直汇总到根。若某个叶子改变，沿路径计算出的根也会改变。要证明一个叶子属于这棵树，只需提供该路径上的兄弟节点，以及每一层的左右顺序。若树比较大，路径长度主要随树高增长，而不需要把全部叶子交给验证者。

在 zRA 中，设备的正确响应不是孤立的。它必须与特定设备身份和特定挑战一起认可，所以论文用 Poseidon 把这三项值绑定成叶子。Poseidon 的选择与电路效率有关：一个在常规 CPU 上很快的哈希算法，未必在算术约束电路中也便宜。零知识友好的哈希可以降低证明电路的开销，但这仍然需要依赖哈希函数的安全性。

这一页的承诺示意图表达的是一般直觉：先固定一个值的承诺，然后用某种方式证明其关联。zRA 在公开验证时并不要求打开承诺、直接展示 response；它通过零知识成员证明实现关联检查。因此，不应理解成验证者最后必须获得所有敏感值，才能确认承诺有效。

最后补充一个部署层面的区别。本文原型使用 Groth16，具有自己的可信初始化要求。协议验证可以透明公开，并不意味着证明系统自动免除了可信初始化。与之分开的另一个准备过程，是为未来响应构建认可承诺树。介绍组件时把这两类准备分开，后面评价信任和成本才不会混淆。

内容出处：原PPT第8页；论文II、IV-A、VI-A及Listing 2

# 第09页 zRA的四个阶段

主讲参考：85秒

## 主讲段（20分钟参考）

这一页把组件放到协议流程中。第一阶段 Setup 由制造商离线计算未来挑战对应的正确响应，并构建承诺树和公开参数。第二阶段按证明周期发布一个全局挑战，未来挑战序列不能一次全部公开。第三阶段设备取得最新挑战，可信测量生成 response，再结合 Merkle 路径生成 zkSNARK 证明并签名提交。第四阶段由公众或合约验证，不仅验证 proof，还检查身份、认可根和最新挑战。这样就把每次证明对特权验证服务的依赖，替换成对公开证据的检查。

## 扩展讲解（按需选用）

Setup 阶段其实做了很多原本会发生在在线验证阶段的准备工作。对于每台设备，制造商知道认可的软件或内部配置，也知道计划使用的挑战序列，因此可以事先计算每个挑战下应该出现的响应。这些响应与设备公钥、挑战一起生成承诺，承诺再进入树。设备以后产生真实响应时，就能够证明自己命中了此前认可的一个承诺。

挑战更新阶段解决的是时间关联。制造商不是在每个周期临时随意选择一个树里没有的新挑战，而是从初始化阶段准备好的序列中逐次发布。如果把所有未来挑战提前公开，攻击者就可能在设备仍处于良好状态时准备后续响应，破坏利用新挑战约束当前测量的目的。因此，公开承诺树与保密未来挑战是可以同时成立的，两者公开的是不同对象。

Attestation 阶段可以分成设备内的两个环节。第一个环节获取真实测量并计算响应，它依靠 ROM 与 MPU 等最小信任锚。第二个环节使用这个响应和路径生成证明，它可以在普通软件层运行。证明完成后，设备还要把证明及相关公开输入绑定到签名或者链上提交身份中，而不只是发送一个脱离上下文的 proof。

Verification 阶段之所以不需要设备秘密，是因为验证者不再计算那个秘密相关的预期响应，而是检查一个关于已认可承诺的证明。不过，这里的“ 已认可 ”必须落实为 validRoots 等检查；挑战新鲜性必须落实为 latestChallenge 检查；设备身份必须与提交者一致。没有这些条件，仅有 verifyProof 返回 true 还不足以接受远程证明。

原 PPT 中“ 公开可见 ”和“ 不公开 ”的对照，应当按原型证明接口理解。root、challenge、设备身份和 proof 参与公开验证；response 与成员路径作为见证，不直接提交为公开输入。私有见证也不等同于必须机密保存的原始状态：论文允许证明计算在不可信软件层完成，安全关键在于真实响应生成的来源与绑定关系。

内容出处：原PPT第9页；论文IV-A、Fig.2、Alg.1 – 3

# 第10页 Setup：未来响应承诺与两层Merkle树

主讲参考：105秒

## 主讲段（20分钟参考）

沿着图从左往右看，设备公钥 pk_k、正确响应 r_i 和挑战 c_i 一起经过 Poseidon，形成一个承诺叶子。每台设备先把自己未来 m 次证明的叶子组成一棵树，各设备的根再作为上层树的叶子，最终得到全局根。若共有 n 台设备，路径总高度约为 ceil(log2 m) 加上 ceil(log2 n)。整个承诺树可以公开，但未来挑战序列仍须保密并逐次发布。它把未来的正确响应提前认可下来，使设备后续生成证明时无需逐次询问制造商；代价是初始化仍需准备约 n 乘 m 个组合。

## 扩展讲解（按需选用）

先说明三个输入为什么都需要。若只承诺 response 而没有设备公钥，其他设备可能尝试使用同一个被认可的响应；若没有 challenge，认可关系就不能区分证明周期。设备身份与挑战共同约束这个响应属于谁、对应哪一次证明。哈希承诺把三项值固定成一个叶子，后续电路又用同样的值重新计算它。

每台设备的下层树包含 m 个叶子。这里 m 表示这台设备规划支持的证明次数，而 n 表示设备数量，二者不能混用。下层路径的长度约为 ceil(log2 m)，上层树把 n 个设备根汇总起来，长度约为 ceil(log2 n)。设备的成员证明需要沿这两层树最终到达同一个全局根。设备自己的根是中间节点，验证认可的是最终的全局根。

用一个数字例子说明规模：如果 n 和 m 都约为一百万，各自的对数高度约为二十，那么总路径高度约为四十。这个例子说明路径长度增长缓慢，但并不说明离线数据规模小。它仍然代表约一万亿个设备—挑战组合；论文对证明阶段的测量不能用来证明这样规模的 Setup 已经在实践中完成。

“ 最终公开的只是 root ”容易产生误解。本页修订后区分了验证锚点与可公开资料：验证关系用全局根锚定认可集合，但树本身含有哈希承诺，也可公开，设备可以从中取得成员路径。未来挑战不能提前全部公开，response 也没有在原型的公开输入列表中直接展示。公开一棵承诺树，不等同于公开承诺背后的所有值。

还有一个实现细节值得准备。论文公式和算法常按公钥、响应、挑战的顺序书写叶子，而 Listing 2 电路实际采用公钥、挑战、响应的输入顺序。这个差异不改变本页“ 绑定三项值 ”的设计思想，但实际生成承诺与验证电路必须使用完全一致的顺序，否则即使数值本身正确也不会得到同一个叶子。汇报时可以保留图中的协议记号，在答辩时说明实现必须对齐。

Setup 完成以后仍有生命周期问题。如果软件版本更新，原来认可的响应和根可能不再适用，需要根据论文提出的更新思路维护新根、挑战与配置，并保证验证者采用当前的认可策略。将离线初始化描述为“ 一次准备 ”，应理解为针对一批设备与规划周期的准备，不宜延伸为设备整个寿命永远无需管理。

内容出处：原PPT第10页；论文IV-A、Fig.3、Alg.1；Listing 2

# 第11页 Attestation：证明关系、公开输入与私有见证

主讲参考：120秒

## 主讲段（20分钟参考）

这一页的重点是公开输入与私有见证。公开输入绑定全局根、当前挑战和设备身份，原型字段是 root、challenge、pubAddr；response 不在公开输入中。私有见证包含 response、Merkle 路径及每层左右方向。设备用公开的证明密钥 d_p 生成 π；这里的证明密钥与上一页设备公钥 pk_k 是两种不同的参数。电路先根据身份、挑战和 response 算出承诺叶子，再沿路径计算根，检查它等于公开 root。图中的设备状态是生成响应的来源，而原始状态的真实测量依靠设备信任锚，并不等于把整台设备的测量过程都放入这段电路。

## 扩展讲解（按需选用）

可以把公开输入 x 理解为这份证明公开声称的上下文。root 告诉验证者，这份证明针对哪个认可集合；challenge 告诉验证者，它对应哪个周期；设备公钥或者原型中的地址告诉验证者，它代表哪台设备。若这些信息没有公开绑定，同一个 proof 就可能被错误地解释成别的设备、别的周期或者别的认可集合下的证据。

私有见证 w 则是证明者用来完成约束计算、但不在公开接口中直接展示的材料。对原型来说，response、pathElements 和 pathIndices 都作为见证。pathElements 是每一层的兄弟节点，pathIndices 决定当前节点应该放在左边还是右边。可以想象在每一层重复一个操作：把当前哈希与兄弟哈希按正确顺序组合，再计算父哈希，最终到达公开的根。

实际电路的第一项关系是 leaf 等于 Poseidon 对设备身份、挑战与响应的哈希；第二项关系是按照成员路径逐层计算父节点；最后一项关系是得到的最终值等于公开 root。π 证明这些计算存在一个满足约束的见证。因此，比较准确的说法是“ 设备知道一个属于认可承诺树的响应 ”，而不是“ π 不依赖任何额外假设就直接证明所有软件永远运行正常 ”。

为什么这样的成员关系能说明软件状态？中间还需要一个条件：响应必须由可靠的状态测量生成。论文将设备划分为信任锚和普通软件层，ROM 保存测量程序，MPU 保护必要的内部状态或存储区域。攻击者可以控制普通软件和网络，但不能随意修改信任锚所保证的测量结果。普通软件层随后取得 response，计算 witness 和 proof。零知识证明不负责纠正伪造的测量来源；它负责防止普通软件层用不满足承诺关系的见证通过检查。

这一分工也解释了为什么证明生成不要求整个过程在安全执行环境内。证明算法和证明密钥是公开的，任何人可以运行算法，但不是任何人都能为指定设备和最新挑战取得满足认可树的响应见证。算法公开与见证关系难以伪造，是可以同时成立的。

原 PPT 原来把 response 写进了公开输入，又把原始状态列为原型见证，容易把一般远程证明概念与本文具体电路混在一起。本轮只在原有公式和说明框中纠正这些值，没有改变证明流程图。页面顶部保留状态来源的示意，讲解时需要说明它先经过可信测量，再成为响应；这张示意图不能代替 Listing 2 的具体输入定义。

最后，私有见证这一术语只说明证明接口不会展示它，不直接等于系统必须把所有见证永久放在安全存储中。论文认为这部分证明计算输入可以在不可信环境中处理，同时原型又把 response 和路径作为零知识见证。汇报时把“对公众不作为公开输入”与“设备内部的保密存储要求”分开，可以避免把零知识性质误解成额外的设备安全硬件要求。

内容出处：原PPT第11页；论文IV-A、Fig.4、Alg.2、Listing 2、附录B

# 第12页 Verification：四项接受条件

主讲参考：90秒

## 主讲段（20分钟参考）

验证算法使用公开验证密钥 vk、公开输入 x 和证明 π。verifyProof 为真，说明电路约束成立，但远程证明的接受还必须检查另外三项条件：设备身份与签名或提交者一致，root 属于认可根集合，challenge 等于最新挑战。身份绑定避免混用其他设备的证明，认可根避免攻击者自己建树，最新挑战避免旧周期证明冒充当前证明。以太坊原型通过 pubAddr 与 msg.sender 比较身份。只有四项条件合在一起，数学证明才成为当前设备、当前周期、当前认可策略下的证据。

## 扩展讲解（按需选用）

第一项是证明系统本身的检查。验证者将 root、challenge、设备身份与 π 一起送入验证函数。如果 proof 针对的是另一组公开输入，或者其约束关系不成立，验证就应失败。这一步负责电路语句的正确性，不负责判断 root 的发行者是否值得认可，也不负责知道哪一个 challenge 才是当前值。

第二项是身份检查。抽象算法使用设备签名，将 π、challenge、设备公钥与根绑定。以太坊原型可以利用交易的提交者身份，再检查 pubAddr 等于 msg.sender。这两种表述分别来自协议层和实现层，不能简单地把原型说成“没有身份认证，只检查 proof”。身份证明和成员证明承担不同工作。

第三项是 root 检查，它经常被忽略。攻击者完全可以挑选一个自己知道的 response，用它造一棵 Merkle Tree，然后对这棵树生成一个正确的成员证明。如果系统允许任意 root，这份 proof 在数学上可能成立，却没有说明设备符合制造商认可状态。所以合约维护有效根集合，验证端必须拒绝不被认可的根。

第四项是挑战检查。latestChallenge 使证据绑定到当前证明周期。过去的证明即使有效，也只说明它针对过去那个挑战成立，不能自动证明当前状态。新挑战还必须按时发布，未来挑战不能过早泄露。否则攻击者可能提前准备本来应该在未来周期才能形成的证据。

还要注意重放问题的范围。对旧周期或者其他设备的重放，身份和挑战检查可以拒绝；对同一设备、同一周期内重复提交完全相同的有效证明，论文中的这组条件并不自动规定必须拒绝。是否记录为同一次状态更新，是否禁止重复交易，需要应用层另行定义。

页面中的 Accept / Reject 是密码学证据是否被接受的结果。应用系统可以在这个结果之上配置业务策略，例如只接受某些版本、某些根或者最近一个周期的状态。公众独立验证的能力来自能够重做这些明确检查，而不是只相信合约显示的一个绿色标签。

内容出处：原PPT第12页；论文IV-A、Alg.3、Listing 1

# 第13页 安全性：攻击方式与安全边界

主讲参考：75秒

## 主讲段（20分钟参考）

这页讨论伪造、重放、消息篡改和 DoS。对于伪造，攻击者需要满足认可树中的响应承诺以及电路约束；对于重放，证明与身份和新挑战绑定；对于篡改，签名及公开输入检查约束提交内容。对于 DoS，zRA 去掉了制造商逐设备处理在线请求的中心瓶颈，但不意味着区块链节点、RPC 服务和设备链路不再受攻击。安全结论依赖密码学、正确初始化与设备最小信任锚；论文不覆盖物理捕获和硬件篡改，也不能保证两次证明之间软件始终安全。

## 扩展讲解（按需选用）

先看伪造。零知识的可靠性要求证明者不能在缺乏有效见证时生成一个被接受的证明。若响应经过可信测量后不对应认可状态，它就无法沿认可承诺树得到指定的 root，除非能够破坏底层哈希或证明系统的安全性。因此，电路约束不是一个独立于哈希、初始化和测量来源的魔法保证，而是这些条件之上的组合保证。

重放防护需要设备身份和时间关联一起讨论。把别的设备 proof 拿来冒充当前设备，身份绑定会产生不一致；把旧挑战下的 proof 拿来冒充当前周期，最新挑战检查会拒绝。但同一周期重复提交的处理仍取决于应用策略。这种区别有助于避免将“ 抵抗重放 ”说成一个不需要任何限定的绝对结论。

消息篡改涉及 proof 以外的字段。如果攻击者能替换 identity、challenge 或 root，而系统只验证签名的一小部分内容，证明语义就可能被错误解释。论文算法把必要公开输入与证明一起签名，原型也使用交易身份及公开参数检查，目的是将完整上下文绑定起来。

DoS 的改善则来自在线角色的改变。制造商不必对每一台设备的证明请求作出响应，验证者也不必访问一个持有设备秘密的中心代理。这减少了一个容易成为拥塞点和单点故障的服务。但设备仍可能无法访问某个节点，某个 RPC 入口仍可能被限流，链上交易也可能受到吞吐与费用制约。部署时可以采用多个节点或其他可用性安排，但论文的协议结论不能被解释为整个系统免疫 DoS。

论文还提出多方准备和发布挑战的阈值机制，减少未来挑战保密与发布可用性对单一参与方的依赖。这里分散的是挑战管理信任，不能自动替代 Groth16 的证明系统初始化，也不能替代设备 ROM、MPU 的信任锚。

最后解释安全时间窗口。一次远程证明对应某次测量及挑战，不等于设备下一秒不会出现新的漏洞或被攻陷。更短的证明间隔可以缩短状态检查间隔，却会增加计算、能耗、网络与链上提交成本。这正好为实验部分提供一个现实问题：我们能够以多高的频率承担这样的证明？

内容出处：原PPT第13页；论文V、IV-B、附录B – C

# 第14页 实验章节过渡

主讲参考：10秒

## 主讲段（20分钟参考）

设计与安全边界介绍完以后，下面进入实验。我们需要检查这套机制是否能在实际平台运行，以及证明生成的时间和资源负担。实验部分既看论文的 Table IV、V、VI，也看原 PPT 保留的复现截图；两类数字的来源与环境会分别说明。

## 扩展讲解（按需选用）

评价实验时可以按三类成本组织问题。第一类是设备端成本，包括 witness 生成、proof 生成、内存、证明密钥和能耗。第二类是验证端成本，包括本地验证计算与链上提交，它们不能混为一个计时指标。第三类是整体基础设施成本，包括离线准备、承诺树存储以及全体设备的证明提交。

本 PPT 保留了原有实验页和截图；新增的详细讲稿用于补充表格的读法和统计边界。我们不把这次整理资料描述成一次新的实验，也不在缺少环境记录的情况下，对复现差异给出唯一原因。

内容出处：原PPT第14页；论文VI – VII

# 第15页 实际应用架构与工作流程

主讲参考：75秒

## 主讲段（20分钟参考）

这一页按设备、Circom、智能合约和 Sepolia 四个模块介绍原型。Circom 描述响应承诺与 Merkle 成员关系并编译为 R1CS；snarkjs 配合 Groth16 准备证明参数和验证器。设备脚本取得公开信息，生成 witness 和 proof，再提交给 Solidity 合约。合约检查设备身份、认可根、最新挑战和 proof，并记录结果。Sepolia 提供公开发布与验证的实验环境。这里的模块编号是架构分工，实际部署需要先准备电路、参数、承诺与合约，设备才能随后提交证明。

## 扩展讲解（按需选用）

设备模块中的 witness 可以理解为完成电路计算所需的变量取值。公开输入与私有输入进入电路以后，计算叶子及各层哈希等中间变量，这个过程产生 witness；随后证明后端利用它生成 π。WIT 和 PRF 两阶段分开计时，是后面读实验表格的基础。

Circom 模块负责描述约束关系，而不是同时承担所有密码学初始化工作。编译电路得到 R1CS 等文件之后，证明系统工具还要生成相应的证明密钥、验证密钥和验证器。原型使用 snarkjs 与 Groth16，因此本轮在原有文字框中补充了 snarkjs 的角色，避免把“ 编译 ”与“ 生成证明参数 ”都归给 Circom。

合约模块把协议接受条件转换成可执行检查。validRoots 对应被认可的承诺根，latestChallenge 对应当前挑战，pubAddr 与 msg.sender 比较对应身份绑定，verifyProof 对应电路证明检查。这些条件要一起成立。链上公开记录使第三方能够查询证据与状态，若有验证能力，也可依据相同公开参数独立检查。

Sepolia 是概念验证部署的公共测试网络。论文展示合约部署、添加根、更新挑战和设备提交证明等交易，说明这些接口能够连接起来运行。它不能直接证明任意规模的真实部署都具有同样费用、时延和吞吐，也不能把测试网络的表现视为所有公开链的保证。

区块链在这里主要提供公开参数、证据发布、验证与归档环境。协议的公开验证思想不必与某条链永久绑定；具体部署也可以研究其他公开数据基础设施。不过，不同基础设施提供的可用性、认可策略和审计能力不同，需要明确替代后仍由谁保证哪些条件。

最后提醒两个计时口径：本地 verifyProof 的计算耗时，不等于设备交易广播到链上最终确认的耗时；证明文件大小，也不等于完整交易包含公开输入、签名和编码之后的传输大小。汇报实验时保持这些单位和对象一致，结果才可比较。

内容出处：原PPT第15页；论文VI-A、Listing 1 – 2、Fig.2、Table III

# 第16页 设备端证明：论文结果与已有复现

主讲参考：150秒

## 主讲段（20分钟参考）

这一页保留论文数据、CPU 信息和原有复现截图。WIT 是 witness 生成时间，PRF 是 proof 生成时间，单位为秒。论文在 h 为四十时，Pi 的两阶段合计约二十六点二秒，i5 笔记本约零点七七秒。原 PPT 的 i7-12700KF 复现每项测试三十次，h 四十时 WIT 约零点二九五一秒，PRF 约零点六四二零秒。可以说 proof 的量级接近论文，而 witness 明显更慢；但 PRF 在不同高度之间并非严格单调。由于硬件、软件后端和计时口径不完全一致，截图不足以确定差异的唯一原因。本次只整理这些已有结果，没有重新运行实验。

## 扩展讲解（按需选用）

读左侧论文 Table V 时，先确定每个单元格中的两个数分别表示什么。以 Raspberry Pi Zero 2 W 为例，树高十时 WIT 为十一点三秒、PRF 为八点四秒；树高四十时 WIT 仍约十一点三秒，而 PRF 增至十四点九秒。两者相加分别约十九点七秒和二十六点二秒。这说明在该实验条件下，witness 阶段比较稳定，证明阶段随路径规模增加总体变慢。

Tinker Board 的 WIT 在不同高度下约三点八至三点九秒，PRF 从六点一六秒增至十一点八秒。i5-12500H 笔记本的 WIT 约零点零一秒，PRF 从零点五一秒到零点七六秒。不同平台相差明显，因此“ 设备证明很快 ”应当同时说明平台、树高以及所包含阶段，不能用笔记本上的表现代表所有 IoT 设备。

再读右侧原有复现表。共同树高十、二十、三十、四十下，WIT 分别约为零点二一九四、零点二二四六、零点二六三二和零点二九五一秒；PRF 分别约为零点五零二三、零点五六六八、零点五四八零和零点六四二零秒。树高三十的 PRF 小于二十，所以这组复现不能被描述为每增加一级树高都会变慢。随机波动、环境与计时方式等都可能影响均值，但没有完整日志时不能确定具体原因。

如果要分析 witness 为什么比论文笔记本更慢，需要检查使用的 witness 生成后端、Node 和 snarkjs 版本、Circom 编译选项、是否把进程启动计入时间、预热方式以及输入文件读取等。CPU 型号本身不能解释所有差异；两台不同处理器之间，也不能只根据型号名称建立严格的速度关系。原 PPT 展示了 CPU 和测试次数，但没有这些完整记录，所以本次适合说“ 观察到差异 ”，不适合说“ 已证明原因 ”。

还可以结合 Table IV 补充资源规模。树高十到四十时，约束数量从二千六百九十一增加到九千九百八十一，witness 从八十六点六 KB 增加到三百二十点九 KB，证明密钥从一点六 MB 增加到六点零 MB。proof 文件大小则约为八百零四至八百零七字节。简洁证明主要帮助验证和传输，并不意味着证明者的计算、witness 或证明密钥也保持不变。

能耗方面，Table VI 给出的 i5 结果为零点六秒、四百七十九毫焦，也就是零点四七九焦耳；Pi 为二十一点八秒、十四点四六焦耳；Tinker 为十一点九秒、五十三点零八焦耳。Tinker 比 Pi 快，但

该表中能耗约为 Pi 的三点六七倍，所以快并不自动意味着省电。不同平台的测量方法也有区别，笔记本使用 powertop 估计，单板机使用 USB 电压、电流测量，比较时应交代口径。

正文还有一个需要审慎处理的数字：其中把笔记本 proof 时间描述为不到四毫秒，与 Table V 的零点五一至零点七六秒不一致。我们以表格中明确列出的计时为主，不把正文的说法混进实验总结。Table VI 的验证小于一毫秒也是验证计算指标，不能当作链上最终确认时间。

这页最终回答的是“ 原型能够运行，但设备负担取决于平台与证明规模 ”。它不是一份所有平台都适用的统一性能承诺，也不是对完整链上系统吞吐的压力测试。

内容出处：原PPT第16页三张截图；论文VI-B – D、Table IV – VI

# 第17页 优势：制造商通信与全网成本

主讲参考：80秒

## 主讲段（20分钟参考）

这一页图中的纵轴是制造商的周期性通信量。传统逐设备发挑战的方式，随着设备数增加，制造商发布量上升；zRA 一个周期公布一个共享挑战，因此制造商发布量与设备数无关。若有 n 台设备、每台证明 m 次，制造商约发布 m 个挑战，全体设备仍约提交 n 乘 m 份证明，离线承诺准备也随 n 乘 m 增长。所以图中的平线不表示整个网络成本为常数。该设计适合设备可以访问公开节点的 Pub/Sub 和 P2P；Mesh 或 Swarm 场景中的实际表现，论文仍留作进一步研究。

## 扩展讲解（按需选用）

可以用一个明确的例子区分统计范围。若有一百万台设备，每台设备计划证明一万次，制造商只要发布一万次全局挑战，而不是为每台设备各发布一万次。这个差异减少了制造商周期性挑战发布的量。但如果每台设备都完成这些证明，设备总体仍会提交一百亿份证明。公共网络、验证基础设施或区块链不能把这部分数据当作不存在。

初始化也有类似区别。每台设备、每个未来挑战都有对应的响应承诺，因此预计算与树叶规模仍然约为 n 乘 m。成员路径只随对数高度增长，proof 又较简洁，这些优点不能代替对全部承诺数据的存储、分发与获取成本进行评估。制造商发布量、设备提交量、树的大小和 proof 大小，是四个不同指标。

论文还讨论了未来挑战的私有存储，它主要随证明次数 m 增长。如果每个挑战三十二字节、一百万个挑战，那么乘法结果是三千二百万字节，约三十二 MB，或者三十点五 MiB。论文中约三 MB 的表述与该乘法不一致，准备答辩时应采用算术结果。这里计算的是未来挑战序列，公开承诺树与路径资料的存储要另计。

为什么 Pub/Sub 和 P2P 更适合？因为设备能够访问公开节点，取得统一挑战和公开资料，而验证者也能从公开环境获得证据。它不需要每个验证者都直接联系制造商，和这类网络的信息传播方式比较相容。对于拓扑动态、连接间歇或者资源极低的 Mesh 和 Swarm 网络，挑战传播、证明提交、节点访问与成员路径获取都可能引入新的负担。

另外，公开链不是无限容量的广播通道。吞吐、费用、节点接口和最终确认都需要单独考虑。将证明固定得很小，或者把制造商的发布从逐设备变成共享发布，可以改善某些成本，却不足以保证一百亿笔提交在任意链上都可行。

这页的结论应表述为一种成本转移与角色改变：制造商的在线逐设备压力下降，设备端承担更多证明计算，而公开基础设施承担证据发布与检查。评价它是否值得，需要结合设备能力、证明频率、公开验证需求和网络可用性一起判断。

内容出处：原PPT第17页；论文VII-B – C、Fig.6 – 7、IX

# 第18页 论文评价与启示章节过渡

主讲参考：10秒

## 主讲段（20分钟参考）

最后进入论文评价。我们会把公开验证与非交互带来的收益，和设备计算、可信测量、初始化及挑战管理的成本放在一起看。这样才能判断 zRA 的贡献，而不是只依据“ 用了零知识证明和区块链 ”这两个标签作评价。

## 扩展讲解（按需选用）

一项协议创新通常需要回答两个问题：它让原来难以实现的能力变得可行了吗，以及它把代价放到了什么地方。对 zRA 来说，新的能力是公众能够直接核验设备证据，代价主要分布在离线承诺准备、设备端证明计算和公开基础设施中。

因此，评价时既要肯定它改变了验证者的权限与在线服务结构，也要说明当前原型不是所有 IoT 场景的统一答案。我们将在最后一页内容中，把这两方面合成一个有边界的结论。

内容出处：原PPT第18页；论文V、VII、IX

# 第19页 结论：贡献、局限与启示

主讲参考：75秒

## 主讲段（20分钟参考）

总结一下，zRA 的主要贡献是把远程证明结果变成任何第三方都能独立检查的公开密码学证据。设备使用响应承诺与 zkSNARK，验证者不需要预共享设备秘密；共享挑战又减少制造商逐设备的在线处理。它仍然依赖可信测量、正确承诺初始化、认可根与挑战发布，Groth16 也有自身初始化要求。原型说明一般 CPU 和部分单板机能够完成证明，但高度受限的 AVR、Cortex-M 等平台仍难承担当前计算。未来可以研究更高效证明后端、硬件加速和安全外包，以及不同网络和更新策略下的实际部署。

## 扩展讲解（按需选用）

可以从三个层面评价贡献。第一是验证权限，公众获得了直接检查证据的能力，不再只接收某个可信方宣告的结果。第二是在线结构，制造商不在每台设备、每个验证者的证明请求关键路径上。第三是证据表达，原来依赖特权参考信息的响应检查，被转换为一个关于认可承诺集合的成员关系证明。这三个变化组合在一起，构成论文从交互走向独立的核心。

同时，信任没有消失。设备测量必须真实，制造商认可的响应必须正确，根集合必须按策略维护，挑战必须新鲜且按时发布，证明系统和签名、哈希等组件必须安全。公开审计让这些条件更容易被明确表达和复核，并不代表条件可以省略。

当前原型的限制首先体现在证明者资源。证明文件约八百字节，并不等于微控制器只需处理八百字节的数据；它还需要 witness、证明密钥和证明计算。论文明确指出极受限 AVR 和 Cortex-M 等平台难以完成当前计算，因此不能把一般 CPU 或单板机的结果推广到所有电池设备。

第二个限制是部署范围。论文认为 Pub/Sub 和 P2P 较适合，而 Mesh、Swarm 仍需要研究；链上吞吐、费用、网络访问与软件更新策略也没有被一个本地计时结果完全回答。Setup 的总体规模同样需要单独评估，不能把制造商挑战发布的 O(m) 当作整个协议的 O(m)。

第三个限制是证明结论的范围。协议提供特定测量、特定挑战与认可状态之间的证据，不负责证明软件从设计上没有漏洞，也不负责抵抗论文未覆盖的物理捕获与硬件篡改。对这类风险，应结合其他系统安全措施，而不能扩大远程证明本身的保证。

论文提出的后续方向包括更适合受限设备的证明实现、其他证明机制、专门硬件以及安全外包。讨论这些方向时，需要继续检查被外包的是哪一段计算、见证由谁持有、响应真实来源是否仍可保证，而不是把外包当成免费取消设备成本的方法。

这篇论文给我们的启示，是把“真实状态测量”和“公开证据验证”清楚分工：前者保留小而明确的信任锚，后者使用公开密码学关系让更多参与者独立复核。对于需要跨组织检查设备状态的场景，这种分工有价值；是否部署，还要看设备和网络能够承担哪些成本。

内容出处：原PPT第19页；论文摘要、IV、VI – VII、IX

# 第20页 致谢与提问

主讲参考：10秒

## 主讲段（20分钟参考）

我们的汇报到这里结束。zRA 让远程证明从依赖特权验证者的交互过程，走向基于公开证据的独立核验，同时保留了需要明确评估的信任与资源边界。谢谢大家聆听，欢迎就协议设计、实验结果和适用场景提问。

## 扩展讲解（按需选用）

如果希望用一句话再次归纳，可以说：设备证明自己知道一个与身份和最新挑战绑定、并属于认可承诺集合的响应，第三方用公开参数检查这份证据。这个结论成立的前提，是可信测量、正确初始化和完整验证条件。

讨论时我们可以围绕三个具体问题展开：透明验证与可信初始化怎样兼容，设备真实测量与零知识电路如何分工，以及制造商通信优势能否弥补设备计算和基础设施成本。逐页讲稿后面准备了对应的答辩备稿，便于根据提问展开。

内容出处：原PPT第20页；汇报结尾

# 附录一：答辩备稿

## 1. 公开透明是不是没有可信初始化？

不是。透明指公众可独立核验协议证据；Groth16 仍有证明系统的可信初始化要求，zRA 还需要准备正确响应承诺以及按时发布挑战。阈值挑战机制分散的是未来挑战的保密与发布可用性信任，不能自动替代证明系统初始化。

## 2. ZK 电路是否重新测量整台设备？

不直接重新测量。原型电路检查 response 的 Poseidon 承诺和 Merkle 成员路径；真实测量与响应生成由 ROM/MPU 信任锚保证。这个分工是判断安全结论的核心。

## 3. 为什么不能让 response 成为公开输入？

Listing 2 的公开列表只有 root、pubAddr、challenge；response 和路径作为见证。公开展示 response 并不是原型的验证接口，也会破坏“只提交公开证明、不展示见证”的讲解。

## 4. 为什么还要检查身份、根和最新挑战？

数学上有效的成员证明不等于业务上认可的设备证明。身份防止混用设备，validRoots 排除攻击者自建树，最新挑战防止用旧周期证据冒充当前证明；同周期内重复提交要另外定义策略。

## 5. 制造商是不是以后完全不需要在线？

不处理逐设备的在线证明请求，但仍需按时发布新挑战，软件更新也需要维护根与参数。论文阈值机制可分散挑战发布的可用性与保密信任，不能把未来挑战一次性提前公开。

## 6. 私有见证是不是必须放在 TEE 中？

不等同。私有见证表示证明接口不公开它；论文认为 response 的哈希形式不直接暴露内部状态，ZK 证明可在不可信软件层生成。真正受保护的是产生真实响应所需的测量程序和内部状态。

## 7. O(m) 的通信优势能否保证大规模链上部署？

不能直接保证。O(m) 指制造商的全局挑战发布量，设备总体提交、离线响应预计算和承诺树仍随 $n \times m$ 增长；链上吞吐、费用、节点访问和存储须另行评估。

## 8. 为什么原PPT复现的 witness 更慢？

目前只能提出候选原因：硬件、witness 后端、Node/snarkjs 版本、编译优化与计时口径。原截图只展示 CPU 型号、测试次数和均值，没有完整环境日志，因此不能断言某个原因，也不能把它视为本次重新复现。

## 9. 软件更新、rollback 与物理攻击怎么办？

论文建议更新二进制、认可根及后续挑战，防止旧软件响应被接受；具体部署还须维护根的认可与更新策略。设备捕获、硬件篡改不在论文覆盖范围，周期性测量也不保证两次证明之间的连续安全。

## 10. 能否用于最小型电池 IoT？

现有概念验证不支持泛化到所有低端设备。论文明确指出 AVR/Cortex-M 等极受限平台难以完成当前证明计算；需要结合频率、电源和资源预算，研究更高效后端、硬件加速或安全外包。

## 11. 为什么可以公开整棵树，却不公开未来挑战？

树公开的是哈希承诺与结构，未来挑战序列则是尚未发布的具体值。设备取得当前挑战后使用对应承诺和路径；提前发布未来挑战可能让攻击者在良好状态下准备未来响应。根是验证认可集合的锚点，公开根不代表只能公开一个根、不能公开树。

## 12. 完整扩展稿应该怎样用于二十分钟汇报？

每页先练习主讲段，并按时间表控制章节节奏；原理推导、额外数字、能耗、信任边界和实现差异作为扩展材料按需使用。详细稿正文约1.6万字符，不能在二十分钟内完整逐字读完。可以在十至十二页保留一两个展开段落，并相应压缩背景复述。

# 附录二：数字与表述核对

## 笔记本 proof 时间

正文 VI-C 有“ 不到4毫秒 ”的表述；Table V 的 i5 PRF 为0.51 – 0.76秒。主讲采用 Table V，h40 两阶段合计0.77秒。

## 未来挑战存储

论文 VII-C 写1,000,000×32 Bytes约3 MB；实际32,000,000 B，约32 MB（30.5 MiB）。公开树存储另计。

## 哈希输入顺序

式(1)/Alg.1写 H^3(pk,r_i,c_i)，Listing 2 实现顺序是(pk,c_i,r_i)。第10页注明协议记号，第11页注明实现顺序，实际Setup与电路必须一致。

## proof 大小的含义

Table IV 报告约805B的原型proof文件大小，不是所有Groth16证明编码的理论最小值。

## 验证 <1 ms

Table VI 是验证计算指标，不能等同于Sepolia端到端确认时间；出块与最终确认不能沿用论文过时概述作为当前链的承诺。

## 复现结果来源

原PPT第16页截图：i7-12700KF，每项30次；本次只整理已有结果。共同高度h10/20/30/40的WIT为0.2194/0.2246/0.2632/0.2951 s，PRF为0.5023/0.5668/0.5480/0.6420 s。

## 能耗口径

Table VI的i5 479mJ换算为0.479J；powertop估计与USB测量的方式不同。Tinker/Pi能耗比53.08/14.46≈3.67，由表中数值计算。

## 通信图的基数

Fig.6/7的图注与正文存在设备数/证明次数基数不一致；第17页采用清楚定义的n=10^6、m=10^4示例，不照抄混淆的基数。